Aula 3:
Recursão e Estruturas Lineares

Pensando em camadas e organizando dados de forma dinâmica

DATA STRUCTURES AND ALGORITHMS | FIAP

FIAP

Professor
Dr. Erick T. Yamamoto

Semestre
2026.2

ALGORITMOS AVANÇADOS



Objetivos da Aula



Dominar o conceito de recursão e sua implementação prática em C.



Compreender o funcionamento da Call Stack (Pilha de Execução).



Saber quando utilizar recursão em vez de iteração (loops).



Introduzir o conceito de Tipos Abstratos de Dados (TADs).



Aprender os fundamentos e a manipulação de Listas Ligadas.

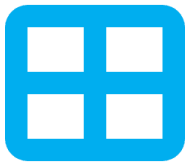
O que é Recursão?

Recursão é um método de resolução de problemas onde a solução depende de soluções para instâncias menores do mesmo problema.

- Na programação, ocorre quando uma **função chama a si mesma**.
- Permite resolver problemas complexos através da **auto-similaridade**.
- Muito comum em algoritmos de ordenação, árvores e grafos.



AUTO-REFERÊNCIA



SUBPROBLEMAS

Os Dois Pilares da Recursão



1. CASO BASE

- É a condição de parada da função.
- Retorna um valor conhecido sem nova chamada.
- **Vital:** Evita o estouro de memória (Stack Overflow).



2. CASO RECURSIVO

- É onde a função chama a si mesma.
- Deve reduzir o problema original.
- **Regra:** Deve sempre caminhar em direção ao Caso Base.

Exemplo Clássico: Fatorial

Definição Matemática

$$n! = n * (n - 1)!$$

Caso Base

$$0! = 1$$

Caso Recursivo

$$n > 0$$

O fatorial de um número é o produto dele pelos seus antecessores até chegar em 1.

```c
// Função recursiva para calcular fatorial
int fatorial(int n) {
    // 1. Caso Base
    if (n == 0) {
        return 1;
    }

    // 2. Caso Recursivo
    return n * fatorial(n - 1);
}
```

Rastreando a Recursão (Tracing)

Visualizando a execução de **fat(3)**

1. Fase de Expansão (Ida)

```
fat(3) = 3 * fat(2)
```

↓

```
fat(2) = 2 * fat(1)
```

↓

```
fat(1) = 1 * fat(0)
```

↓

```
fat(0) = 1 (Caso Base)
```

2. Fase de Substituição (Volta)

```
fat(3) = 3 * 2 = 6
```

↑

```
fat(2) = 2 * 1 = 2
```

↑

```
fat(1) = 1 * 1 = 1
```

↑

```
Retorna 1
```

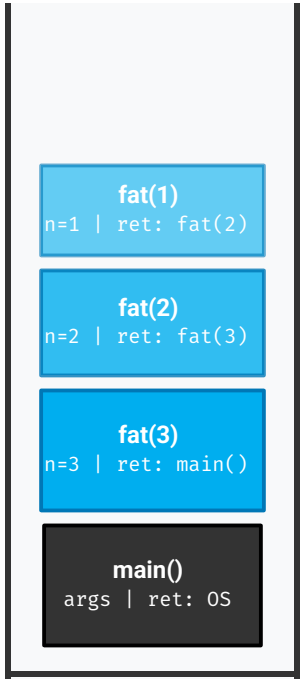
A Call Stack (Pilha de Execução)

O que é? Uma região da memória RAM que armazena informações sobre as funções ativas no programa.

Stack Frame: Cada chamada de função cria um bloco contendo:

- Parâmetros da função
- Variáveis locais
- Endereço de retorno (onde continuar)

LIFO: Segue a lógica *Last-In, First-Out*. A última função chamada é a primeira a ser finalizada.



REPRESENTAÇÃO DA PILHA

O Perigo: Stack Overflow

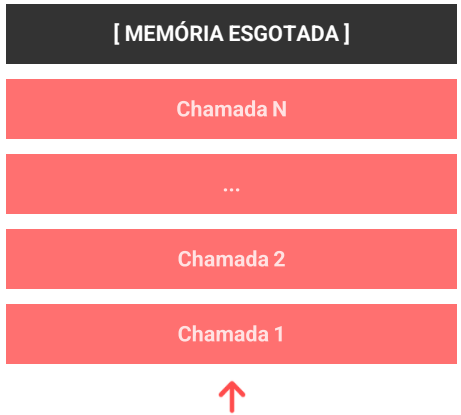
Esgotamento da Pilha

Ocorre quando o programa tenta usar mais memória na Call Stack do que o sistema operacional permitiu.

```c
// Exemplo de Erro Fatal
int infinito(int n) {
   // Esqueceu o Caso Base!
    return n + infinito(n + 1);
}
```

- Ausência de Caso Base
- Caso Base nunca alcançado
- Recursão profunda demais (N muito grande)

"Cada chamada consome memória. Sem parada, a memória acaba."



Recursão vs. Iteração (Loops)

Embora ambos possam resolver os mesmos problemas, a escolha entre eles impacta diretamente o uso de recursos do sistema.

Característica	Recursão	Iteração (Loops)
Controle	Chamadas de função	Estruturas for / while
Memória	Usa a Call Stack (O(n))	Espaço constante (O(1))
Velocidade	Overhead de chamadas	Execução direta e rápida
Código	Mais limpo e elegante	Pode ser mais verboso

💡Quando escolher?

Use **Iteração** para tarefas simples e repetitivas. Use **Recursão** para problemas com estrutura auto-similar, como navegação em árvores, grafos ou algoritmos de "dividir e conquistar".

Recursão de Cauda (Tail Recursion)

Recursão Comum

```c
int fat(int n) {
   if (n == 0) return 1;

   return n * fat(n - 1);
}

// O computador precisa "lembrar" de multiplicar por N após o retorno.
```

Recursão de Cauda

```c
int fat_cauda(int n, int acc) {
   if (n == 0) return acc;

   return fat_cauda(n - 1, n * acc);
}

// A chamada recursiva é a ÚLTIMA operação. O resultado é passado via acumulador.
```

Otimização: Compiladores modernos podem transformar a **Recursão de Cauda** em um loop simples internamente, eliminando o risco de *Stack Overflow* e economizando memória da pilha.

Atividade em Sala 1: Fibonacci

Desafio: Sequência de Fibonacci

Implemente uma função recursiva em C que receba um número `n` e retorne o n-ésimo termo da sequência de Fibonacci.

Caso Base: Se `n <= 1`, retorne o próprio `n`.

Caso Recursivo: Retorne a soma de `fib(n-1) + fib(n-2)`.

Teste sua função no `main()` com valores pequenos (ex: 5, 10).

Reflexão: O que acontece se você tentar calcular `fib(50)`?

DICA: Desenhe a árvore de chamadas para fib(4) e veja quantas vezes a mesma conta é repetida!

Resumo da Parte 1: Recursão



Auto-referência

Recursão ocorre quando uma função resolve um problema chamando a si mesma para instâncias menores.



Os Dois Pilares

Todo algoritmo recursivo precisa de um **Caso Base** (parada) e um **Caso Recursivo** (fluxo).



Call Stack

O sistema gerencia as chamadas empilhando **Stack Frames** na memória RAM até atingir o caso base.



Segurança

Cuidado com o **Stack Overflow**! Garanta que o caso base seja sempre alcançável.

Tipos Abstratos de Dados (TAD)

O que é um TAD?

É um modelo que define um conjunto de dados e as operações que podem ser realizadas sobre eles, sem especificar como serão implementados.

Encapsulamento: Os detalhes internos de armazenamento são ocultados do usuário final.

Interface: O usuário interage apenas com as funções públicas (ex: inserir, remover, buscar).

Exemplo: TAD Lista

OPERAÇÕES DEFINIDAS

- criar_lista()
- inserir_elemento(x)
- remover_elemento(x)
- buscar_elemento(x)
- esta_vazia()

"A mesma interface pode ser implementada usando Vetores ou Listas Ligadas."

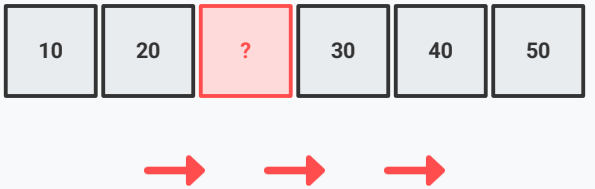
Limitações dos Vetores (Arrays)

Tamanho Estático: O tamanho deve ser definido na compilação ou no início da execução. Não pode crescer conforme a demanda.

Desperdício de Memória: Se alocarmos espaço para 1000 itens e usarmos apenas 10, o restante da memória fica inutilizado.

Inserção/Remoção Lenta: Operações no meio do vetor exigem o deslocamento de todos os elementos subsequentes.

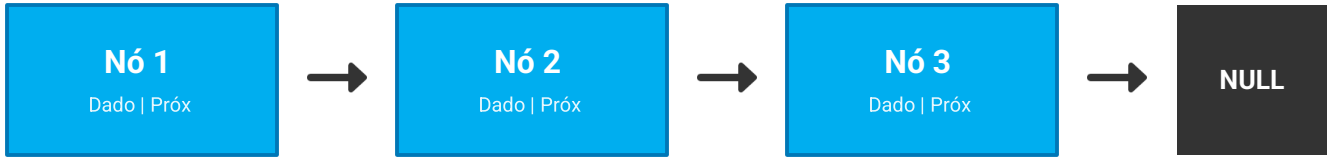
O Custo do Deslocamento (Shifting)



COMPLEXIDADE O(N) PARA INSERÇÃO

Para inserir um elemento na posição 2, precisamos "empurrar" todos os outros para a direita.

Introdução às Listas Ligadas



Dinamicidade: A lista cresce e diminui em tempo de execução, sem tamanho fixo.

Memória Não-Contígua: Os elementos podem estar em qualquer lugar da memória RAM.

Conexão via Ponteiros: Cada elemento conhece apenas o endereço do próximo.

Inserção Eficiente: Adicionar elementos não exige o deslocamento de toda a estrutura.

A Unidade Básica: O Nó (Node)



Campo de Dado

Armazena a informação real (inteiro, caractere, struct, etc.).



Campo de Ponteiro

Armazena o endereço de memória do **próximo nó** da lista.

A conexão entre esses nós forma a corrente que chamamos de Lista Ligada.

```c
// Definição do Nó em C
struct Node {
    int dado; // A informação
    struct Node* proximo; // O link
};
```

Note a autorreferência: o ponteiro é do mesmo tipo da struct.

Alocação Dinâmica (malloc)

Memória Heap

Diferente da Stack, o **Heap** permite alocar memória que persiste além do escopo da função atual.

Função malloc()

Solicita ao sistema operacional um bloco de bytes. Retorna um ponteiro para o início desse bloco.

sizeof: Sempre use `sizeof(struct Node)` para garantir que o espaço alocado seja exato para a estrutura.

NULL Check: Sempre verifique se o retorno é `NULL` (falha de memória).

```c
// Criando um novo nó dinamicamente
struct Node* novoNo;

// Aloca espaço para 1 nó
novoNo = (struct Node*) malloc(sizeof(struct Node));


if (novoNo == NULL) {
    printf("Erro: Memória insuficiente!\n");
    exit(1);
}

// Atribuindo valores
novoNo->dado = 10;
novoNo->proximo = NULL;
```

Operação: Inserção no Início

- 1. Alocar memória para o novo nó.
- 2. Atribuir o valor ao campo de dado.
- 3. Apontar o 'proximo' do novo nó para o início atual.
- 4. Atualizar o ponteiro de início para o novo nó.

```c
void inserirInicio(Node** head, int valor) {
   Node* novo = malloc(sizeof(Node));
   novo->dado = valor;
   novo->proximo = *head;
   *head = novo;
}
```

Complexidade: O(1)

Diagrama de Reapontamento



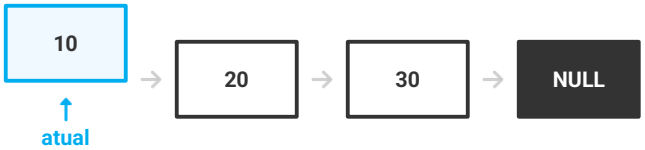
Diferente dos vetores, não precisamos "empurrar" ninguém. Basta mudar o endereço do ponteiro.

Operação: Percurso (Traversal)

O Ponteiro Auxiliar

Nunca use o ponteiro **head** para caminhar na lista, ou você perderá a referência do início para sempre!

Criamos um ponteiro temporário (ex: `atual`) que começa no `head` e pula de nó em nó até encontrar o `NULL`.



Complexidade: O(n)

```c
// Função para imprimir a lista
void imprimirLista(Node* head) {
    Node* atual = head; // Auxiliar

    while (atual != NULL) {
        printf("%d -> ", atual->dado);
        atual = atual->proximo; // Pulo
    }

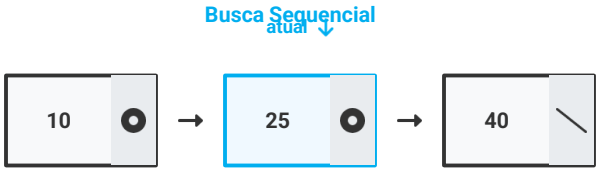
    printf("NULL\n");
}
```

Operação: Busca na Lista

1. Iniciar um ponteiro auxiliar no **head** .
2. Enquanto o ponteiro não for **NULL** :
   - 2.1. Se o dado for igual ao alvo, retorne o nó.
   - 2.2. Senão, avance para o próximo nó.
3. Se sair do loop, o valor não existe.

```
Node* buscar(Node* head, int alvo) {
   Node* atual = head;
   while (atual != NULL) {
      if (atual->dado == alvo) return atual;
      atual = atual->proximo;
   }
   return NULL;
}
```

Complexidade: O(n)



Diferente dos vetores, não há acesso direto por índice.
É preciso "caminhar" pela lista nó por nó.

Operação: Remoção de um Nó

1. Localizar o nó a ser removido e seu **anterior**.
2. O anterior deve apontar para o **próximo** do nó removido.
3. Liberar a memória do nó removido com `free()`.

```
// Exemplo simplificado de remoção
Node* temp = anterior->proximo;
anterior->proximo = temp->proximo;
free(temp); // Crucial!

// Caso seja o primeiro nó (head):
Node* aux = *head;
*head = (*head)->proximo;
free(aux);
```

Complexidade: O(n) para busca

Lógica de "Bypass" e Free



O nó anterior "pula" o nó removido.
Atenção: *Nunca esqueça o `free()` para evitar vazamento de memória (Memory Leak).*

GERENCIAMENTO DE MEMÓRIA

Comparativo: Vetor vs. Lista Ligada

Operação / Atributo	Vetor (Array)	Lista Ligada
Acesso Aleatório	**O(1) - Imediato**	**O(n) - Precisa percorrer**
Inserção no Início	**O(n) - Exige deslocamento**	**O(1) - Apenas ponteiros**
Inserção no Meio	**O(n) - Exige deslocamento**	**O(n) - Exige busca**
Memória	Estática / Contígua	Dinâmica / Espalhada
Overhead	**Nenhum (apenas dados)**	**Ponteiros extras por nó**

USE VETORES QUANDO:

O tamanho é conhecido, o acesso aleatório é frequente e a economia de memória (sem ponteiros) é prioritária.

USE LISTAS QUANDO:

O tamanho é imprevisível, inserções/remoções no início são constantes e a memória RAM está fragmentada.

Atividade em Sala 2: Manipulação de Nós

Desafio: Construção Manual

Sem usar funções de inserção, crie uma lista ligada com 3 elementos diretamente na função `main()`.

Passo 1: Declare 3 ponteiros do tipo `struct Node*` (ex: n1, n2, n3).

Passo 2: Aloque memória para cada um usando `malloc` e atribua valores aos campos de dado.

Passo 3: Faça o encadeamento: `n1->proximo = n2`, `n2->proximo = n3` e `n3->proximo = NULL`.

Passo 4: Imprima os valores acessando a partir do primeiro nó (ex: `n1->proximo->dado`).

DICA: Ao final, não esqueça de usar a função free() para liberar a memória de cada nó alocado!

Desafio para Casa e Próxima Aula

Desafio Prático

Implemente uma função recursiva em C que inverta a ordem dos elementos de uma lista ligada.

- Use o Git Flow para organizar seu código.
- Documente a complexidade Big O da sua solução.
- Submeta o link do repositório no portal.

Próxima Semana

Na Semana 4, exploraremos duas estruturas lineares fundamentais baseadas em restrições de acesso:

Pilhas (Stacks) - LIFO

Filas (Queues) - FIFO

Preparem-se para entender como o sistema gerencia processos e impressões!